

CICLO DE COMPLEMENTACIÓN

LICENCIATURA EN CIENCIA DE DATOS

CLCD 182 - SEMINARIO DE
ACTUALIZACIÓN I

Clases

Semana 06





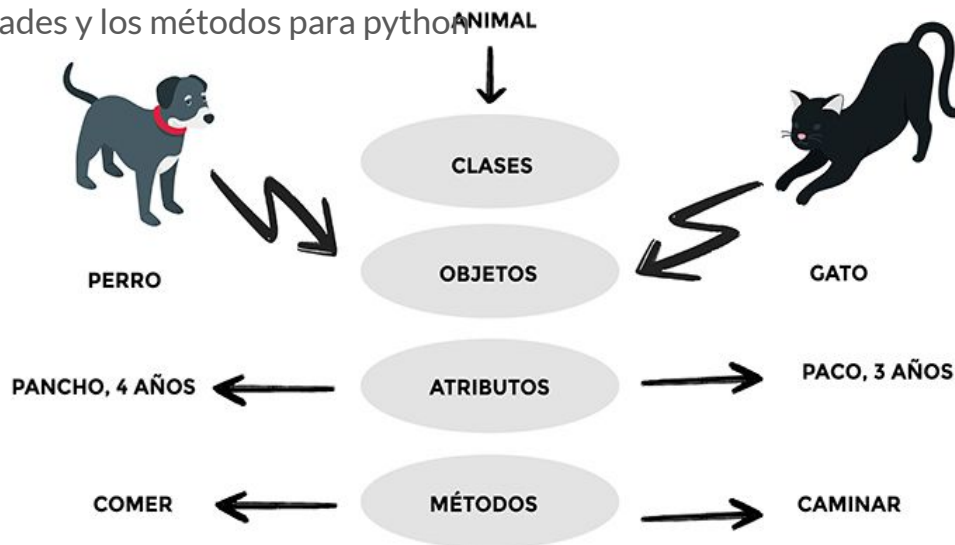
Temas a Desarrollar

- Programación orientada a objetos
- Tipos de métodos
- Herencia
- Abstracción
- Encapsulamiento
- Polimorfismo
- Cohesión
- Acoplamiento
- Decorador property

Programación orientada a objetos



podes crear una imagen sencilla para explicar la programación orientada a objeto, como las clases, los objetos, las propiedades y los métodos para python





Programación orientada a objetos



Definiendo una clase, se encuentra en un archivo o papel

```
# Creando una clase vacía
class Perro:
    pass
```

Creamos un objeto de la clase Perro,
se genera en ejecución

```
# Creamos un objeto de la clase perro
mi_perro:Perro = Perro()
print('Se crea un objeto de la clase perro')
print(type(mi_perro))
```

```
Se crea un objeto de la clase perro
<class '__main__.Perro'>
```



Programación orientada a objetos

Tipos de atributos:

- **Atributos de instancia:** se crean en tiempo de ejecución, en el constructor.
- **Atributos de clase:** se crean y pertenecen a la clase.

```
class Perro:
    # Atributo de clase
    especie = 'mamífero'
    # Nombre de la especie

    # El método __init__ es llamado al crear el objeto
    def __init__(self, nombre:str, raza:str):
        """ Constructor de la clase perro

        Args:
            nombre (str): Nombre del perro
            raza (str): Raza del perro
        """
        print(f"Creando perro {nombre}, {raza}")

    # Atributos de instancia
    self.nombre = nombre
    self.raza = raza
```



Tipos de métodos

- De instancia

```
def metodo(self, arg1:str, arg2:str) -> Tuple:
```

```
class = ClaseTestNormal([argumento_instancia='constructor'])  
class.metodo(arg1='método_arg1', arg2='método_arg2')
```

- De clase

```
@classmethod
```

```
def metododeclase(cls, arg1:str, arg2:str) -> Tuple:
```

```
ClaseTestClassMethod.metododeclase(arg1='método_arg1', arg2='método_arg2')
```

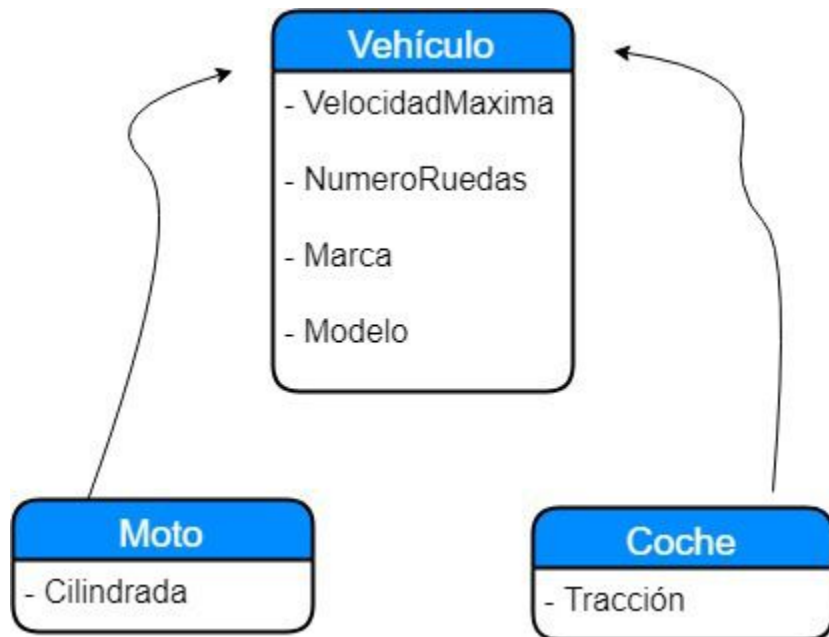
- Estáticos

```
@staticmethod
```

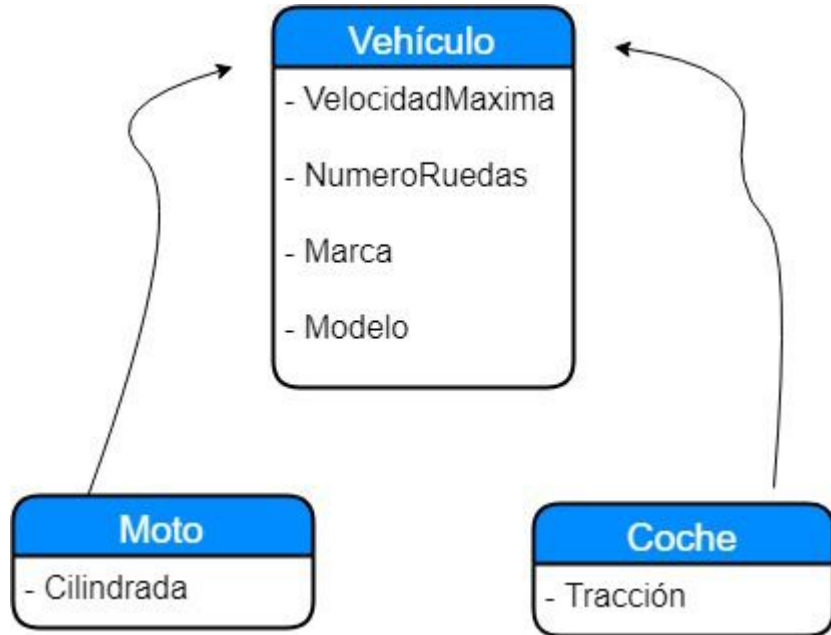
```
def metodoestatico(arg1:str, arg2:str) -> str:
```

```
ClaseTestStaticMethod.metodoestatico(arg1='método_arg1', arg2='método_arg2')
```

Herencia



Herencia



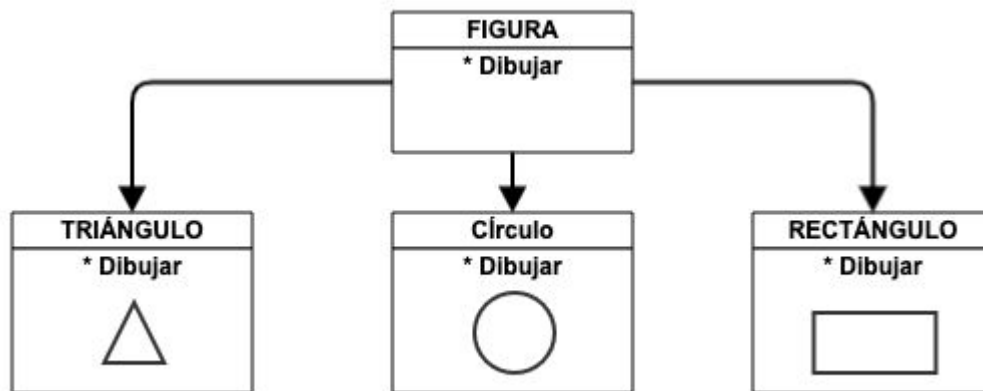
```
# Definimos una clase padre
class Vehiculo:
    pass

# Creamos una clase hija que hereda de la padre
class Moto(Vehiculo):
    pass

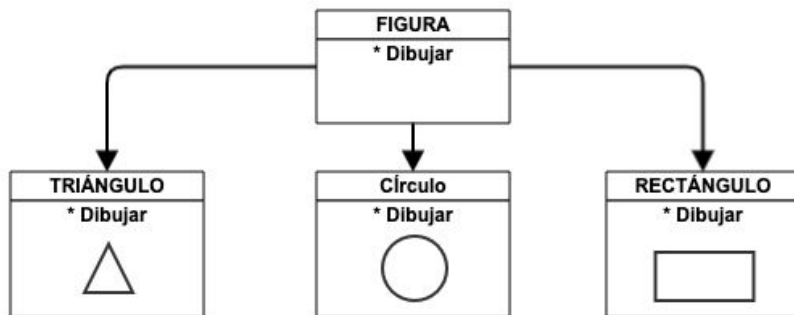
# Creamos una clase hija que hereda de la padre
class Coche(Vehiculo):
    pass
```

Abstracción

Ocultar la complejidad



Abstracción



```
from abc import ABC

class Figura(ABC):
    def dibujar(self):
        pass
```

```
from abc import abstractmethod
from abc import ABCMeta

class Figura(metaclass=ABCMeta):
    @abstractmethod
    def dibujar(self):
        pass

class Triangulo(Figura):
    def dibujar(self):
        pass

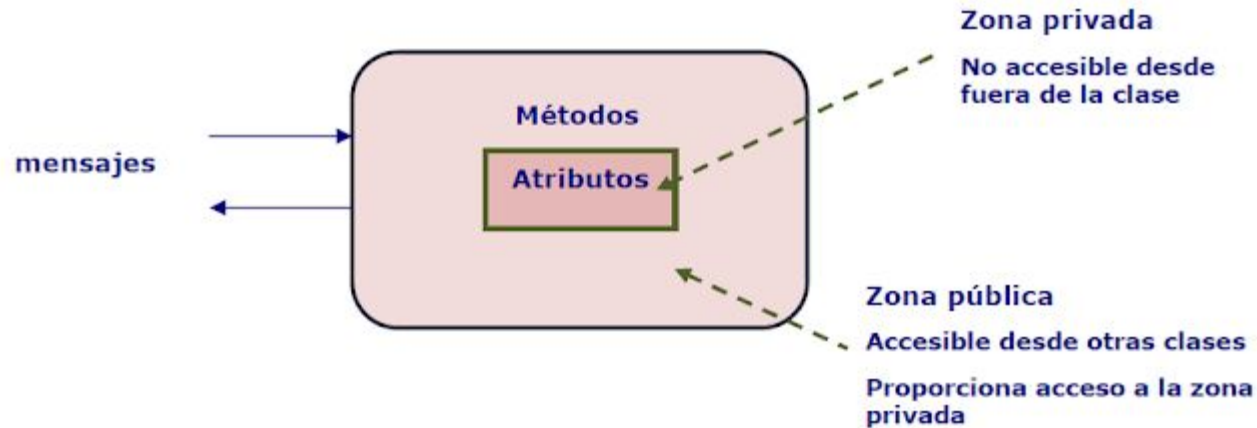
class Circulo(Figura):
    def dibujar(self):
        pass

class Rectangulo(Figura):
    def dibujar(self):
        pass
```

Encapsulamiento



Ocultar los estados internos de los objetos

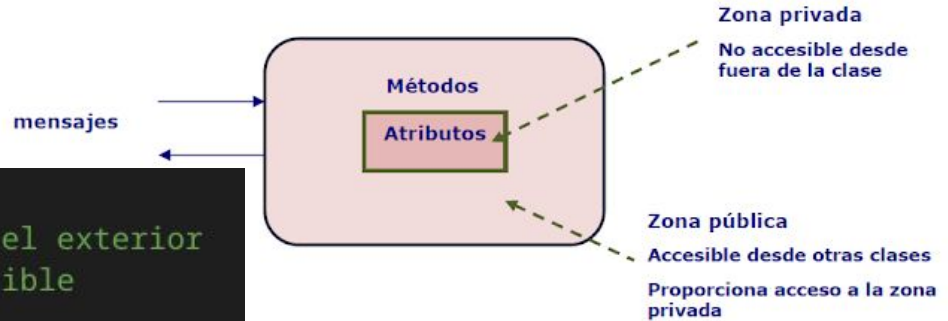


Encapsulamiento

```
class Clase:
    atributo_clase = "Hola"    # Accesible desde el exterior
    __atributo_clase_privado = "Hola" # No accesible

    # No accesible desde el exterior
    def __mi_metodo_privado(self):
        print("Dentro del método privado")
        self.__variable = 0

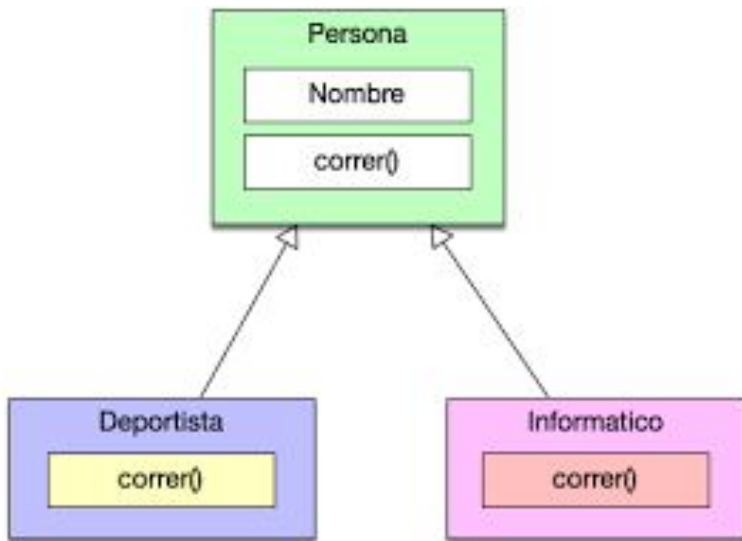
    # Accesible desde el exterior
    def metodo_normal(self):
        print("Dentro del método normal", end=' - ')
        # El método si es accesible desde el interior
        self.__mi_metodo_privado()
```



Polimorfismo



Las clases poseen un método que se llama de la misma forma pero se comporta de acuerdo a la clase.

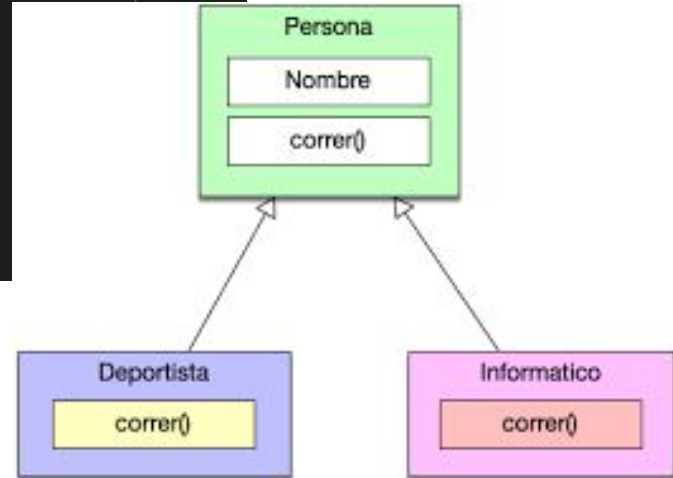


Polimorfismo

```
class Persona:
    nombre:str = ''
    def comer(self):
        raise NotImplementedError("Este método debe ser implementado por la subclase")

class Deportista(Persona):
    def comer(self):
        print("Saludable!")

class Informatico(Persona):
    def comer(self):
        print("No tiene problemas!")
```





Cohesión

Grado de relación entre los elementos de un módulo.

Tipos de cohesión:

- **Débil:** indica que la relación entre los elementos es baja. Es decir, no pertenecen a una única funcionalidad.
- **Fuerte:** indica que existe una alta relación entre los elementos existentes dentro del módulo.

Porqué deseamos una cohesión debil:

- a. Reducir la complejidad del módulo, ya que tendrá un menor número de operaciones.
- b. Se podrá reutilizar los módulos más fácilmente.
- c. El sistema será más fácilmente mantenible.



Acoplamiento

Mide la dependencia entre dos módulos distintos de software

Tipos de acoplamiento:

- **Débil:** indica que no existe dependencia de un módulo con otros.
- **Fuerte:** indica que un módulo tiene dependencias internas con otros.

Porqué no queremos un acoplamiento Fuerte

- a. Un cambio puede generar un efecto mariposa.
- b. Reduce la reusabilidad.



Decorador property

Se utiliza para encapsular las propiedades.

Posee las siguientes opciones:

- **Getter:** permite acceder al valor de la propiedad.
- **Setter:** permite setear un valor en una propiedad.
- **Deleter:** se ejecuta cuando se elimina una propiedad.



Decorador property

```
class Clase:
    def __init__(self, mi_atributo):
        """ Constructor de la clase

        Args:
            mi_atributo: Valor del atributo
        """
        self.__mi_atributo = mi_atributo

    @property
    def mi_atributo(self):
        return self.__mi_atributo

    @mi_atributo.setter
    def mi_atributo(self, valor):
        if valor != "":
            print("Modificando el valor")
            self.__mi_atributo = valor
        else:
            print("Error está vacío")

    @mi_atributo.deleter
    def mi_atributo(self):
        del self.__mi_atributo
        print("El atributo fue borrado")
```